

Langa Workspace: AI Integration & Google Cloud Architecture

A Technical Portfolio Specification of Serverless Dual-Client RAG & Sovereign Data Residency | Ntivo EdTech

CONFIDENTIAL COMMERCIAL ARCHITECTURE SPECIFICATION (ABSTRACTED FOR PORTFOLIO USE)

Langa Workspace is an AI-native educational SaaS platform engineered to automate South African Department of Basic Education (DBE) CAPS-aligned lesson plans and assessments. Built as a decoupled microservices application on Google Cloud Platform (GCP), the platform combines a Next.js presentation tier with a FastAPI containerized engine. This specification details how generative AI models, retrieval-augmented generation (RAG), and domestic POPIA data residency are seamlessly integrated across serverless cloud infrastructure.

1. AI Architecture Overview & The Location Paradox

Langa Workspace utilizes a **Decoupled Dual-Client Routing Pattern** in `ai_service.py` that separates knowledge discovery from generative reasoning. This design explicitly resolves GCP's *'Location Paradox'*: guaranteeing strict POPIA data sovereignty by storing all educator profiles and PII in the `africa-south1` (Johannesburg) region, while routing non-PII search vectors and context prompts to multi-regional Vertex AI engines in Europe (`eu`, `europa-west1`, `europa-west4`) under a strict Zero-Learner-Data policy.

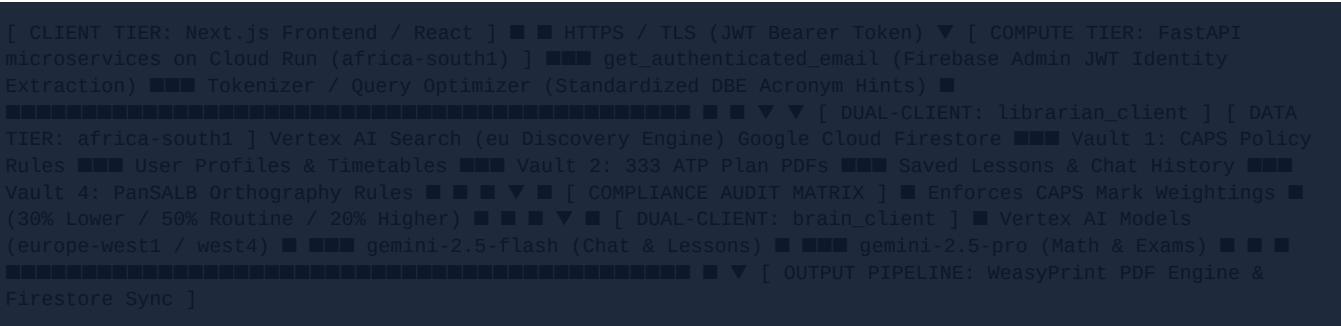


Figure 1: End-to-End Serverless AI & Cloud Data Flow Architecture

2. End-to-End Application-to-AI Data Flow

- **Step 1: Authenticated Ingress & Token Decoding:** Educator sends curriculum generation parameters via Next.js UI. FastAPI's `get_authenticated_email` dependency interceptors decode the JWT session token via Firebase Admin SDK, validating identity without trusting body payloads.
- **Step 2: Tokenizer & Query Optimization:** System tokenizers strip 'Grade' labels to raw integers and map subject names to government-standard acronyms (e.g., 'Physical Sciences' → Phys Sci) to build precise search hints.
- **Step 3: Quad-Vault Grounding (`librarian_client`):** Issues semantic queries to Vertex AI Search Quad-Vault datastores in `eu` to extract exact Annual Teaching Plan (ATP) week/term guidelines.
- **Step 4: Compliance Audit Matrix:** Evaluates retrieved policy paragraphs and calculates mandated cognitive mark distributions (30% Lower Order, 50% Routine, 20% Higher Order) prior to model invocation.
- **Step 5: Generative Model Inference (`brain_client`):** Dispatches augmented context prompts to `gemini-2.5-pro` or `gemini-2.5-flash` in `europa-west1`.
- **Step 6: Parsing & Multi-Tenant Persistence:** Output passes through recursive brace-matching parsers (`get_matching_brace`) and backslash cleaners in `pdf_service.py` to convert LaTeX math strings into native HTML/CSS for WeasyPrint PDF compilation. Documents save to Firestore in `africa-south1` tagged with `school_id`.

3. Vertex AI Models & Quad-Vault RAG Components

AI / RAG Component	Deployment Region	Technical Function & Scope
gemini-2.5-flash	europa-west1	High-throughput model for daily lesson plans, conversational chat, and Agent Nano cron tasks.
gemini-2.5-pro	europa-west4	High-reasoning model for STEM exam papers, complex LaTeX math rendering, and multi-part memorandums.
CAPS Policy Vault (Vault 1)	eu (Discovery Engine)	Holds national policy frameworks and cognitive mark weighting directives (30/50/20).
ATP Vault (Vault 2)	eu (Discovery Engine)	Indexes 333 official Annual Teaching Plan PDFs and CC-BY STEM textbooks for weekly pacing.
Orthography Vault (Vault 4)	eu (Discovery Engine)	Official Pan South African Language Board (PanSALB) rulebooks for 11 official languages.

4. Document Processing, Security & Storage Infrastructure

- **WeasyPrint LaTeX Cleaning Engine:** Replaced brittle client-side math renderers with a backend Python parser (pdf_service.py) using get_matching_brace to convert LaTeX equations into native HTML/CSS elements for A4 PDF compilation.
- **In-Memory FastAPI Policy Proxy:** Heavy administrative school policy PDFs (10MB+) are uploaded via FastAPI multipart buffers and streamed directly to Google Cloud Storage (langa-policy-vault) using StreamingResponse, avoiding client-side signed URLs.
- **Zero-Trust Keyless Service Account Impersonation:** Backend containers use the impersonated_credentials wrapper to impersonate langa-executor at runtime, completely eliminating static .json key files from repositories.
- **POPIA & Zero-Learner-Data Enforcement:** Strict data schema validation forbids collecting student PII. Educator records reside exclusively in Firestore instances hosted in africa-south1 (Johannesburg).

5. Logging, Observability & Autonomous Cron Execution

Google Cloud Logging & Monitoring: Captures structured logs across Cloud Run containers, Cloud Build pipelines, and Vertex AI latency metrics. Error monitoring tracks rate-limit spikes (429 Too Many Requests) and invokes exponential backoff retry decorators.

Autonomous Agent Nano Worker: Managed by **Google Cloud Scheduler**, issuing OIDC-authenticated HTTP requests every **Friday at 1:00 PM** to trigger /api/v1/agent/cron/weekly-prep, pre-drafting weekly lesson plans for all teachers based on assigned timetables.

6. Summary of Documented Lead AI Engineering Contributions

- ✓ **Monolith-to-Microservices Modernization:** Migrated legacy monolithic code to a high-performance Next.js 14 + FastAPI serverless microservices platform on Cloud Run.
- ✓ **Solving the GCP Location Paradox:** Architected the dual-client routing pattern (brain_client vs. librarian_client) that preserves POPIA data residency in africa-south1 while utilizing European Vertex AI infrastructure.
- ✓ **The Compliance Audit Matrix Engine:** Designed the 3-step RAG pipeline that programmatically forces generative AI models to adhere to national CAPS cognitive mark distributions (30% Lower / 50% Routine / 20% Higher).
- ✓ **Autonomous Background Worker (Agent Nano):** Engineered agent.py and integrated Cloud Scheduler OIDC triggers to automate weekly lesson plan prep.
- ✓ **Keyless IAM & In-Memory Storage Proxy:** Implemented Zero-Trust GCP Service Account impersonation and built an in-memory FastAPI upload/download streaming proxy.